

Trabalho Prático 3 — MercadoDCC

Leandro Miranda Santos

Matrícula: 2025050091

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil
leandromrs@ufmg.br

1 Introdução

Este trabalho implementa o *MercadoDCC*, um sistema de comércio eletrônico em linha de comando capaz de cadastrar usuários e produtos, registrar compras e reposições de estoque, e responder a consultas multiatributo sobre todas essas entidades. A consulta combina filtros sobre diferentes atributos por interseção de conjuntos de identificadores, utilizando índices invertidos que mapeiam cada valor de atributo para o conjunto de entidades que o possuem.

O núcleo do projeto é manter, para cada atributo buscável, uma estrutura que associa valores a conjuntos ordenados de identificadores. Uma consulta recupera um conjunto por filtro e combina os resultados. Foram implementados dois backends para esse mapa valor→identificadores: um **vetor ordenado** de pares (chave, conjunto), favorecendo localidade de referência; e uma **tabela hash** com encadeamento, favorecendo inserções e buscas em tempo amortizado constante. Os pontos extras incluem operadores booleanos nas consultas e buscas por faixas numéricas em atributos selecionados. O histórico agregado de compras, usado na segunda linha das consultas LU e LP, também possui duas implementações comparáveis: listas esparsas (**HistoricoLista**) e matrizes densas (**HistoricoMatriz**).

2 Método

O sistema foi escrito em C++11, com um par de arquivos por tipo abstrato de dados não genérico e templates header-only para estruturas parametrizadas. Memória dinâmica é gerenciada manualmente. Identificadores de entidades são atribuídos sequencialmente a partir de zero e coincidem com o índice no vetor principal de armazenamento, o que permite recuperação direta após a consulta.

2.1 Fluxo funcional

O programa lê linhas da entrada padrão. Comandos de cadastro (U, P) criam entidades e atualizam os índices invertidos correspondentes. Comandos de transação (C, R) alteram estoque e históricos; compras inválidas por estoque insuficiente são rejeitadas sem modificar o estado. Consultas (LU, LP, LC, LR) tokenizam os filtros restantes da linha, avaliam a expressão, por interseção implícita ou por operadores booleanos, e imprimem os registros encontrados em ordem crescente de identificador.

2.2 Tipos abstratos de dados

- **Usuário, Produto, Compra e Reposição**: encapsulam os atributos das entidades e a formatação de saída. Usuário e produto são imutáveis após cadastro, exceto a quantidade em estoque do produto. Compra só é instanciada quando há estoque suficiente.
- **ConjuntoIds**: conjunto ordenado de inteiros sem duplicatas, implementado sobre um vetor dinâmico. Suporta interseção, união e complemento em tempo linear no tamanho dos operandos, por varredura casada sobre sequências ordenadas.
- **Vetor**: TAD vetor genérico, com alocação contígua, capacidade inicial de 8 posições (sem construir elementos), método **reservar** e duplicação geométrica na expansão.
- **MapaVetor** (variante padrão): mapa valor→ConjuntoIds mantido como vetor ordenado por chave. Busca por busca binária; inserção desloca elementos adjacentes.
- **MapaHash** (variante experimental): tabela hash com listas encadeadas por bucket, redimensionada quando a carga atinge metade da capacidade.
- **Índice invertido** e **Índice invertido inteiro**: índices invertidos selecionado em tempo de compilação, um para chaves textuais e outro para inteiras.
- **ÍndiceOrdenado**: árvore balanceada por valor numérico, usada para faixas inclusivas em idade, preço, quantidade em estoque e timestamp. Permite inserção, remoção e busca por intervalo.
- **Histórico de compras (HistoricoCompras)**: agrega totais usuário↔produto em compras válidas, para a segunda linha de LU/LP. Duas variantes selecionadas em compilação:
 - **HistoricoLista** (padrão): para cada usuário e cada produto, uma lista ordenada de pares (identificador, quantidade). Memória proporcional ao número de pares distintos; inserção $O(k)$ na lista do par.
 - **HistoricoMatriz**: duas matrizes espelhadas $U \times P$ de inteiros, com atualização $O(1)$ por par e varredura contígua na impressão. Reserva $U \cdot P$ células por visão, favorecendo localidade espacial quando o histórico é denso.
- **Sistema**: orquestra entidades, índices, validação de estoque e avaliação de consultas.

2.3 Consultas multiatributo

Na versão base, filtros consecutivos na linha são interpretados como conjunção (AND implícito): o resultado é a interseção dos conjuntos retornados por cada filtro. No ponto extra de operadores booleanos, a linha pode conter NOT, AND e OR, com precedência NOT > AND > OR. A expressão é convertida para notação polonesa reversa e avaliada com pilha de conjuntos; NOT calcula o complemento em relação ao universo da entidade consultada.

No ponto extra de faixas, dois números consecutivos após um atributo numérico elegível definem um intervalo inclusivo. Atributos com suporte a faixa: idade, preço e quantidade, timestamp. A busca por faixa percorre apenas nós do índice ordenado cujas chaves podem intersectar o intervalo.

2.4 Variantes de compilação

- **MapaVetor (make all)**: entradas contíguas, boa localidade espacial na busca binária e na varredura dos conjuntos de identificadores.

- **MapaHash** (`make hash-build`): lookup amortizado $O(1)$, porém nós de colisão dispersos no heap, com menor localidade de referência.
- **HistoricoLista** (`make all`): backend padrão do histórico; combina com MapaVetor ou MapaHash nos índices.
- **HistoricoMatriz** (`make hist-matriz-build`): matrizes densas; binário `tp3_hist_matriz.out` para comparação isolada de tempo e memória do histórico.

As variantes de índice e de histórico são independentes: o histórico pode ser lista ou matriz enquanto os índices invertidos permanecem em vetor ou hash. Apenas o backend selecionado em cada dimensão muda; a lógica de negócio e o formato de saída permanecem idênticos.

3 Análise de Complexidade

3.1 Notação

Sejam n_u, n_p, n_c, n_r : números de usuários, produtos, compras e reposições cadastrados (também U, P, C, R); k chaves distintas em um índice invertido ou árvore ordenada; m tamanho de um **ConjuntoIds**, sendo ids associados a uma chave ou resultado parcial; f filtros atômicos em uma consulta; q : itens por compra ou reposição; r : cardinalidade do resultado; d_u : produtos distintos comprados por u ; d_p : usuários distintos que compraram p ; $\bar{m}$ tamanho médio dos conjuntos por chave em um índice.

3.2 Estruturas elementares

3.2.1 Vetor<T>

Armazenamento contíguo; todo **Vetor** nasce com buffer de `CAPACIDADE.INICIAL`= 8 (memória bruta, sem objetos construídos); expansões dobram a capacidade.

- **Acesso operator[]**: $O(1)$.
- **Inserção push_back**: $O(1)$ amortizado; $O(n)$ no pior caso isolado, pois realocação e cópia dos n elementos.
- **Reserva reservar(n)**: $O(n)$ quando n excede a capacidade atual.
- **Memória**: $O(n)$ objetos; capacidade pode exceder n por reserva ou slots ociosos.

Entidades (**Usuario**, **Produto**, **Compra**, **Reposicao**) residem em vetores indexados pelo id, permitindo recuperação pós-consulta em $O(1)$ por identificador.

3.2.2 ConjuntoIds

Vetor ordenado sem duplicatas; base das consultas multiatributo.

- **Inserção/remoção ordenada**: $O(m)$ posição mais deslocamento.
- **Membership contem**: $O(\log m)$ por busca binária.
- **Interseção, união, complemento**: $O(m_a + m_b)$ por varredura com two-pointer em vetor ordenado.
- **Interseção múltipla**: $O(f \cdot m)$ no pior caso com f operandos de tamanho $O(m)$.
- **Memória**: $O(m)$ inteiros contíguos por conjunto.

3.3 Índices invertidos (MapaVetor e MapaHash)

Cada índice mapeia valor de atributo para **ConjuntoIds**. Os wrappers **IndiceInvertido** e **IndiceInvertidoInt** delegam ao backend de compilação.

3.3.1 MapaVetor

Vetor ordenado de entradas (**chave**, **ConjuntoIds**).

- **Busca**: $O(\log k)$ (busca binária em `buscar_pos`).
- **Inserção** em chave existente: $O(\log k + m)$.
- **Inserção** de chave nova: $O(k + m)$ e $O(\log k)$ para localizar, mais $O(k)$ para deslocar entradas.
- **Memória**: $O(k \cdot \bar{m})$ mais chaves; entradas contíguas, sem overhead de nó.

3.3.2 MapaHash

Tabela hash com encadeamento; rehash quando `_tamanho` $\geq$ `_capacidade/2`.

- **Busca/inserção** em chave existente: $O(1)$ esperado; $O(k)$ no pior caso.
- **Inserção** de chave nova: $O(1)$ amortizado; rehash ocasional em $O(k)$.
- **Memória**: $O(B + k)$ buckets ($B = O(k)$) mais, por chave, nó encadeado, chave e **ConjuntoIds** overhead de ponteiro `prox` por entrada.

3.4 IndiceOrdenado (faixas numéricas)

Árvore AVL por valor (**idade**, **preco**, **qtd**, **timestamp**), independente do backend vetor/hash dos índices de igualdade.

- **Inserção/remoção** de id em chave: $O(\log k + m)$.
- **Busca pontual**: $O(\log k)$.
- **Busca por faixa** `buscar_faixa`: $O(\log k + t + s)$, com t chaves na faixa e s ids agregados (pior caso $O(k)$).
- **Memória**: um nó AVL por valor distinto; ponteiros dispersos no *heap*.

3.5 Histórico agregado de compras

3.5.1 HistoricoLista (padrão)

Listas ordenadas de **ParIdQtd** por usuário e por produto.

- **Registro de compra** (q itens): $O(\sum_{i=1}^q (d_{u,i} + d_{p,i}))$; $O(q \cdot d_{\max})$ no pior caso.
- **Impressão LU/LP**: $O(d_u)$ ou $O(d_p)$ por entidade exibida.
- **Memória**: $O(\sum_u d_u + \sum_p d_p)$ esparsa.

3.5.2 HistoricoMatriz (experimental)

Doas matrizes espelhadas $U \times P$ de inteiros.

- **Cadastro** de usuário: $O(P)$; de produto: $O(U)$.
- **Registro de compra**: $O(q)$ atualizações $O(1)$ cada.
- **Impressão LU/LP**: $O(P)$ ou $O(U)$ por resultado, varre linha inteira mesmo com poucas células não nulas.

- **Memória:** $O(U \cdot P)$ inteiros por matriz; total $O(U \cdot P)$ nas duas visões.

3.6 Operações do sistema

3.6.1 Cadastro de usuário (U)

Uma inserção em `_usuarios` ($O(1)$ amortizado) e sete atualizações de índice, com cinco mapas textuais/inteiros, `id` e idade em `IndiceOrdenado`. Com `MapaVetor`, cada índice custa até $O(k_i)$; com `MapaHash`, $O(1)$ amortizado. **Tempo:** $O(\sum_{i=1}^7 \text{custo_inserir}(k_i))$. **Memória:** $O(n_u)$ na entidade mais $O(\sum_{\text{índices}} k_i \bar{m}_i)$.

3.6.2 Cadastro de produto (P)

Análogo: sete índices: nome, categoria, marca, condição, id, preço, quantidade. Memória adicional do histórico: **Lista** $O(1)$; **Matriz** $O(U)$ ao expandir colunas.

3.6.3 Compra (C) e reposição (R)

Para q pares produto- i , quantidade: validação $O(q)$; reindexação de estoque $O(q \cdot (\log k_q + m))$; histórico **Lista** $O(q \cdot d)$ ou **Matriz** $O(q)$; índices da transação $O(q)$ inserções nos mapas.

3.6.4 Consultas (LU, LP, LC, LR)

- **Filtro de igualdade:** lookup $O(\log k)$ ou $O(1)$ amortizado.
- **Filtro por faixa:** `buscar_faixa` no `IndiceOrdenado`.
- **Conjunção** de f filtros: f lookups mais $f - 1$ interseções, $O(f \cdot m)$.
- **Booleanos:** AND/OR em $O(m_a + m_b)$; NOT complemento em $O(n)$ sobre o universo $\{0, \dots, n - 1\}$.
- **Impressão:** $O(r)$ acessos às entidades; em LU/LP, mais $O(r \cdot d)$ (**Lista**) ou $O(r \cdot \max(P, U))$ (**Matriz**).

3.7 Memória global do sistema

Decomposição do uso reportado por MEM:

- **Entidades:** $O(n_u + n_p + n_c + n_r)$ nos vetores principais.
- **Índices invertidos:** soma de $O(k \cdot \bar{m})$ por atributo; na hash, overhead de buckets e ponteiros.
- **Índices ordenados:** $O(k)$ nós AVL por atributo numérico elegível.
- **Histórico:** esparsa $O(\sum d_u + \sum d_p)$ ou densa $O(U \cdot P)$.

O consumo cresce linearmente com cadastros nos índices; quadraticamente apenas com **HistoricoMatriz** e U, P grandes.

3.8 Comparação entre variantes

Operação	MapaVetor	Hash
Busca	$O(\log k)$	$O(1)$ amort.
Ins. chave nova	$O(k)$	$O(1)$ amort.
Ins. chave exist.	$O(\log k + m)$	$O(1) + O(m)$
Memória/chave	menor	maior

Tabela 1: Backends de índice invertido.

Operação	Lista	Matriz
Atualiz./item	$O(d)$	$O(1)$
Impressão LU/LP	$O(d)$	$O(P)$ ou $O(U)$
Cadastro U/P	$O(1)$	$O(P) / O(U)$
Memória	$O(\sum d)$	$\Theta(U \cdot P)$

Tabela 2: Variantes de histórico.

O **MapaVetor** privilegia memória previsível e localidade; a **hash** reduz cadastros quando k é grande. No histórico, a **matriz** troca memória por atualização constante; a **lista** adequa-se ao perfil esparsa. Nenhuma variante domina em todas as dimensões.

3.9 Análise de Localidade de Referência

A localidade espacial depende de quão contíguos estão os acessos em memória durante cada operação.

- **ConjuntoIds:** identificadores em array contíguo; interseção e união percorrem índices consecutivos, favorecendo cache e prefetch.
- **MapaVetor:** pares (chave, conjunto) em bloco contíguo; a busca binária acessa uma região compacta do vetor a cada passo.
- **MapaHash:** buckets formam array contíguo, mas cada colisão exige seguir ponteiros para nós alocados separadamente, padrão de acesso mais disperso.
- **Vetor de entidades:** após obter identificadores ordenados, acessar registros por índice é $O(1)$; identificadores consecutivos implicam acessos adjacentes no vetor de entidades.
- **ÍndiceOrdenado** (faixas): árvore com nós dispersos; busca por faixa percorre apenas subárvores relevantes, mas ainda com saltos por ponteiro.
- **HistoricoLista:** cada lista de pares ocupa bloco contíguo; inserção desloca elementos adjacentes, mas a impressão percorre apenas entradas existentes.
- **HistoricoMatriz:** a linha $[u][0..P-1]$ ou $[p][0..U-1]$ é um vetor contíguo de inteiros; a varredura na segunda linha de LU/LP apresenta forte *localidade espacial* com padrão diagonal/sequencial em traços de memória, porém percorre células zeradas quando o histórico é esparsa.

Medições com Valgrind Cachegrind e mapas Lackey (Seção 5.2.9) quantificam esses efeitos: em cargas com predominância de consultas, espera-se que o MapaVetor apresente traços mais diagonais; em cadastros massivos, a hash pode compensar pelo menor número de comparações por inserção, apesar da pior localidade espacial nos nós encadeados.

4 Estratégias de Robustez

Além das escolhas assintóticas discutidas na Seção 3, o sistema adota medidas que preservam invariantes de dados e evitam estados inconsistentes mesmo sob entradas válidas mas exigentes.

- **Validação prévia de estoque em compras:** `registrar_compra` percorre todos os pares produto-quantidade antes de alterar qualquer estrutura. Se algum item não tem estoque suficiente, imprime **C INV** e retorna sem criar registro, sem decrementar estoque, sem atualizar índices nem histórico. A operação é atômica do ponto de vista do usuário.
- **Ids sequenciais como índices diretos:** cada entidade recebe identificador igual ao próximo índice livre no vetor principal (`_prox_id_*`). Após a consulta, o acesso `_usuarios[id]`, `_produtos[id]`, etc. é $O(1)$ e dispensa mapa auxiliar `id`→`posição`, desde que os índices invertidos permaneçam sincronizados no cadastro.
- **ConjuntoIds ordenado e sem duplicatas:** `inserir` ignora ids repetidos e mantém o vetor ordenado, garantindo que interseção, união e complemento por two-pointer produzam resultados corretos e em ordem crescente, como exige a saída das consultas.
- **Retorno seguro em filtros inválidos:** atributos desconhecidos, comandos fora de contexto ou faixas em atributos não elegíveis retornam `ConjuntoIds::conjunto_vazio()`, singleton estático reutilizável. Consultas sem resultado imprimem **LU VAZIO**, **LP VAZIO**, etc., em vez de

falhar silenciosamente.

- **Reindexação consistente de quantidade:** compras e reposições alteram o estoque do produto e, em seguida, removem o id do valor antigo e inserem no valor novo no índice `_idx_p_qtd`, evitando que o mesmo produto permaneça associado a quantidades obsoletas.
- **Expansão coordenada do histórico:** ao cadastrar usuário ou produto, `HistoricoLista` e `HistoricoMatriz` expandem os buffers necessários (`ao_cadastrar_usuario/ao_cadastrar_produto`) antes de registrar compras, de modo que acessos por id não ultrapassem limites após cadastros válidos.
- **Parsing restrito de faixas numéricas:** dois números consecutivos só formam intervalo inclusivo quando o atributo é elegível e ambos os tokens passam em `eh_numerico`; caso contrário, o segundo token é interpretado como filtro separado, alinhado à gramática do enunciado.
- **Ordenação determinística em transações:** `Compra` e `Reposicao` ordenam os pares produto–quantidade por id de produto no construtor, garantindo saída estável em LC/LR independentemente da ordem de leitura na entrada.
- **Rehash controlado na tabela hash:** quando a carga atinge metade da capacidade, a tabela dobra de tamanho, reinsere todas as entradas em buckets novos e só então libera o array antigo, mantendo tempo amortizado e evitando degradação por colisões excessivas.
- **Gerenciamento manual de memória:** classes com alocação dinâmica (`Compra`, `Reposicao`, nós da `MapaHash`) definem destrutor, construtor de cópia e operador de atribuição, liberando buffers internos e evitando aliasing na cópia de vetores de `Compra/Reposicao` armazenados no `Sistema`. O programa foi testado com valgrind sem vazamentos reportados.

5 Análise Experimental

5.1 Configuração

Os experimentos foram executados em Linux (x86_64), com entradas sintéticas geradas deterministicamente. O `Vetor<T>` genérico inicia com `CAPACIDADE_INICIAL=8` e dobra de tamanho sempre que necessário.

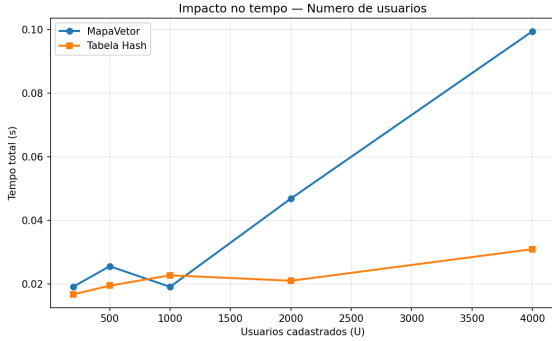
Cada configuração compara `MapaVetor` e `MapaHash` sob a **baseline** fixa: 1000 usuários, 1000 produtos, 200 compras, 200 reposições, 100 consultas de cada tipo (LU/LP/LC/LR), 2 filtros por consulta, 3 itens por transação. Em cada experimento, *apenas um parâmetro* varia; os demais permanecem na baseline.

Tempo total e memória estimada foram coletados ao final de cada carga. Contadores internos registram comparações em `ConjuntoIds` e inserções nos índices invertidos. Cada ponto corresponde a uma execução completa.

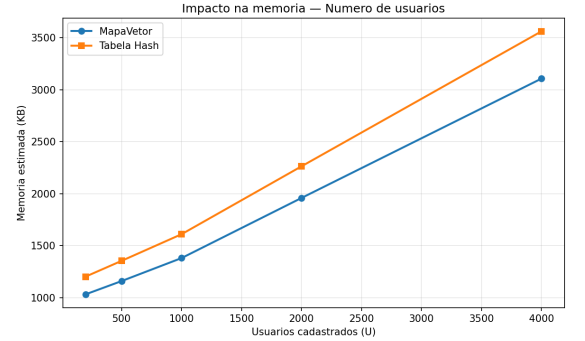
5.2 Resultados

5.2.1 Impacto do número de usuários

$U \in \{200, 500, 1000, 2000, 4000\}$, demais parâmetros fixos.



(a) Tempo total.



(b) Memória estimada.

Figura 1: Impacto do número de usuários cadastrados.

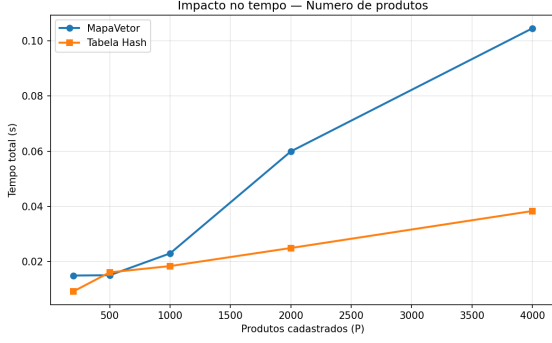
Teoria: cada U executa sete inserções em índices (cadastro de usuário). Com `MapaVetor`, chave nova custa $O(k_i)$ por deslocamento; com `MapaHash`, $O(1)$ amortizado (Tabela 1). Tempo total $O(U \cdot \sum_i \text{custo.inserir}(k_i))$; memória $O(U + \sum_i k_i \bar{m}_i)$.

Empírico: a curva de tempo cresce aproximadamente linear em U . Inserções em índice sobem de $\approx 12\,200$ ($U = 200$) para $\approx 38\,800$ ($U = 4000$), compatível com trabalho extra por usuário. A hash vence a partir de $U \geq 2000$ e com $U = 4000$ leva 0,031 s contra 0,099 s do vetor ($\approx 3,2\times$), confirmando o termo $O(k)$ do `MapaVetor` quando k é grande.

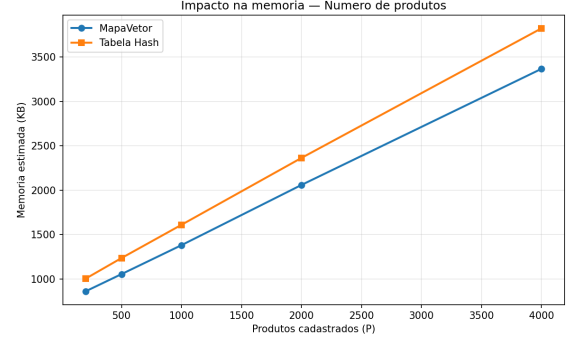
Memória: crescimento quase linear em U . Na baseline, $\approx 1,41$ MiB (vetor) e $\approx 1,65$ MiB (hash, +17%), refletindo overhead de buckets e ponteiros previsto na Tabela 1.

5.2.2 Impacto do número de produtos

$P \in \{200, 500, 1000, 2000, 4000\}$, $U = 1000$ fixo.



(a) Tempo total.



(b) Memória estimada.

Figura 2: Impacto do número de produtos cadastrados.

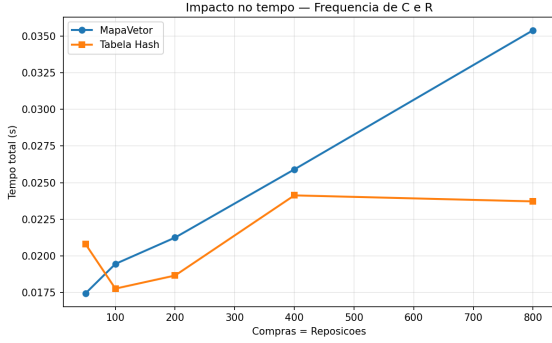
Teoria: cadastro de produto tem a mesma forma assintótica (sete índices por P). O vetor `_produtos` usa expansão geométrica a partir de capacidade 8.

Empírico: curvas análogas o experimento 1, tempo cresce com P e a hash vence em escala ($P = 4000$: 0,105s vs. 0,038s). Para $P \leq 1000$, `_produtos` ainda sofre realocações ($8 \rightarrow 16 \rightarrow \dots \rightarrow 1024$), mas o deslocamento $O(k)$ do `MapaVetor` nos índices permanece dominante.

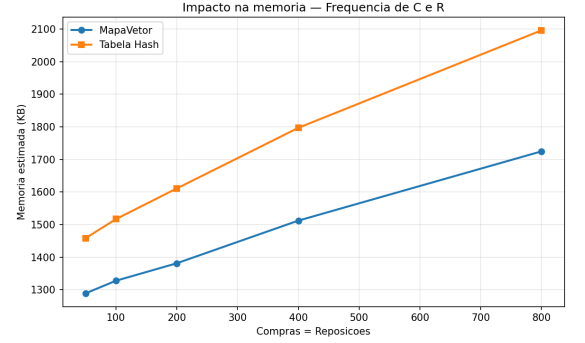
Memória: na baseline ($P = U = 1000$), totais idênticos a E01 ($\approx 1,41$ MiB / $\approx 1,65$ MiB), pois a carga de entidades e índices é simétrica.

5.2.3 Impacto da frequência de compras e reposições

$C = R \in \{50, 100, 200, 400, 800\}$, estoque inicial suficiente para compras válidas.



(a) Tempo total.



(b) Memória estimada.

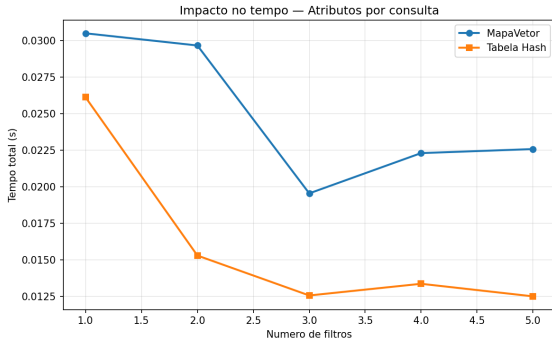
Figura 3: Impacto do número de compras e reposições.

Teoria: cada compra/reposição válida custa $O(q)$ validações, $O(q \cdot \log k_q)$ na reindexação de estoque (`IndiceOrdenado`), $O(q \cdot d)$ no histórico lista e $O(q)$ inserções nos índices da transação. O total entra linearmente em C , mas cadastros e consultas fixos na baseline somam termo constante grande.

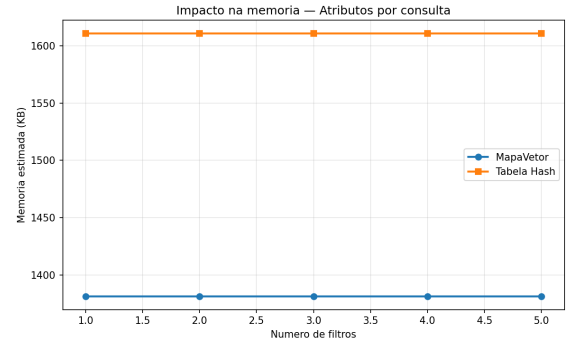
Empírico: de $C = R = 50$ a 800, o tempo sobe de $\approx 0,017$ s para $\approx 0,035$ s (vetor) — sublinear na variação de C , pois os ≈ 2000 cadastros e 400 consultas fixos diluem o incremento $O(C)$ previsto. Memória cresce levemente com C (histórico esparsa e índices de transação), em linha com $O(C)$ entradas adicionais.

5.2.4 Impacto do número de atributos por consulta

Número de filtros $f \in \{1, 2, 3, 4, 5\}$, consultas exatas.



(a) Tempo total.



(b) Memória estimada.

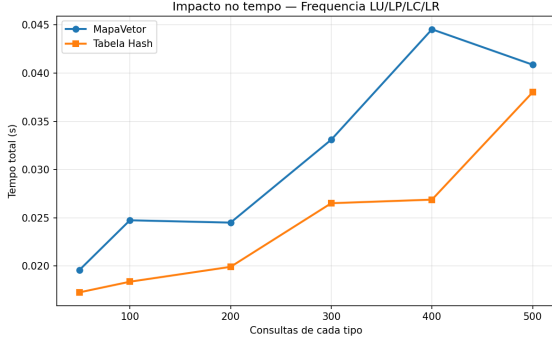
Figura 4: Impacto do número de filtros por consulta.

Teoria: consulta com f filtros exatos executa f lookups ($O(\log k)$ ou $O(1)$ cada) e $f - 1$ interseções em `ConjuntoIds` — $O(f \cdot m)$ no pior caso.

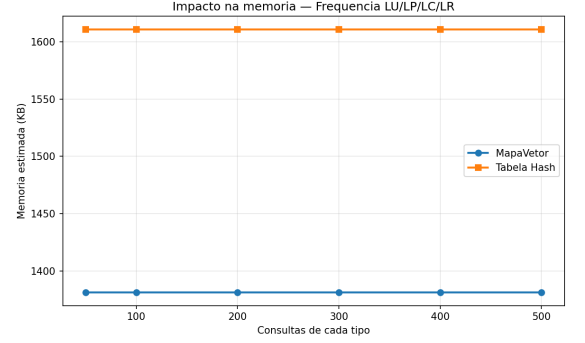
Empírico: o tempo permanece estável (0,013–0,030 s) quando f vai de 1 a 5. Isso *não* contradiz a análise: após o primeiro filtro, m encolhe rapidamente na carga sintética, de modo que as interseções subsequentes operam sobre conjuntos pequenos — o termo $O(f \cdot m)$ fica barato frente ao custo fixo $O(U + P)$ de montar os índices. Comparações em `ConjuntoIds` variam pouco (0 com $f = 1$ até $\approx 71\,700$ com $f = 5$), corroborando conjuntos intermediários pequenos. Memória invariante, como esperado.

5.2.5 Impacto da frequência de consultas

100 consultas de cada tipo (LU/LP/LC/LR) variando de 50 a 500.



(a) Tempo total.



(b) Memória estimada.

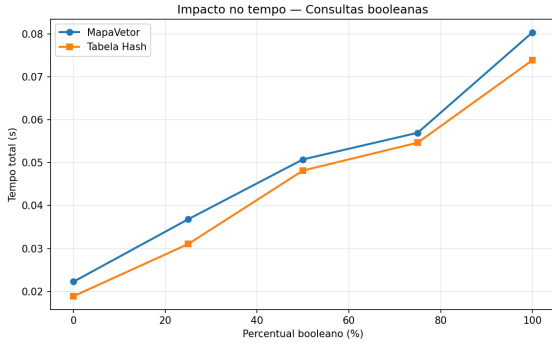
Figura 5: Impacto da frequência de consultas por tipo.

Teoria: o custo marginal de cada consulta é f lookups mais combinações $O(m)$ em `ConjuntoIds`; repetir consultas escala linearmente, enquanto o backend do mapa afeta apenas o lookup.

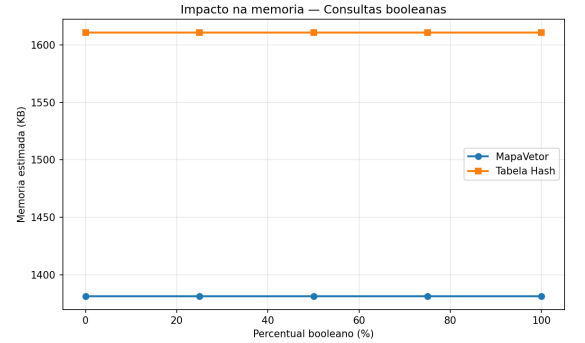
Empírico: ao quintuplicar consultas por tipo (100 $\rightarrow$ 500), as comparações sobem de $\approx 48\,000$ para $\approx 252\,000$ ($\approx 5,2\times$) e o tempo de 0,025 s para 0,041 s ($\approx 1,6\times$). A sublinearidade do tempo relativa às comparações ocorre porque o termo constante de cadastro ($O(U + P)$ índices) não escala com consultas. Vetor e hash permanecem próximos: o gargalo é `ConjuntoIds::intersect`, comum às duas variantes (Tabela 1). Memória constante ($\approx 1,41$ MiB).

5.2.6 Impacto de consultas booleanas

Percentual de consultas com operadores NOT/AND/OR: {0, 25, 50, 75, 100}%.



(a) Tempo total.



(b) Memória estimada.

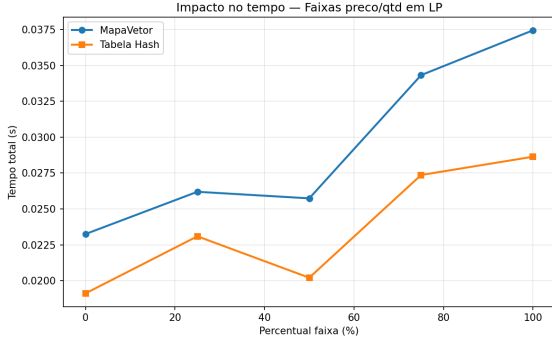
Figura 6: Impacto do percentual de consultas booleanas.

Teoria: operadores booleanos acrescentam união e complemento em `ConjuntoIds`, cada um $O(m_a + m_b)$ ou $O(n)$ sobre o universo da entidade. O complemento (NOT) constrói $\{0, \dots, n - 1\}$ em $O(n)$ antes de subtrair.

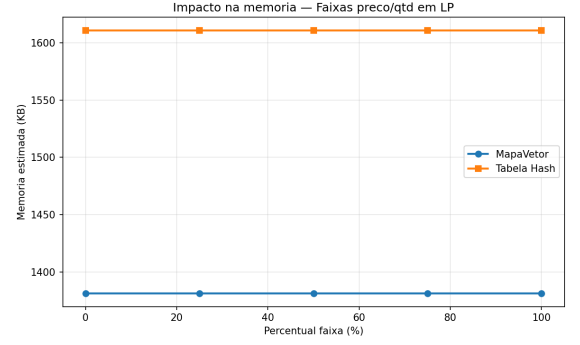
Empírico: de 0% a 100% de consultas booleanas, o tempo no vetor cresce de 0,022 s para 0,080 s ($\approx 3,6\times$); comparações, de $\approx 48\,000$ para $\approx 107\,000$ ($\approx 2,2\times$). O tempo escala mais que as comparações porque NOT materializa conjuntos grandes mesmo quando poucos ids satisfazem a consulta — cenário previsto para complemento em $O(n)$. A hash acompanha (0,074 s em 100%), pois o custo extra está na combinação de conjuntos, não no lookup do mapa.

5.2.7 Impacto de faixas em preço e quantidade

Percentual de consultas LP com faixa inclusiva em preço ou quantidade.



(a) Tempo total.



(b) Memória estimada.

Figura 7: Impacto de consultas por faixa de preço e quantidade.

Teoria: faixa substitui lookup pontual $O(\log k)$ por `buscar_faixa` no `IndiceOrdenado` — $O(\log k + t + s)$ com t chaves na faixa. O backend `MapaVetor/MapaHash` não participa desse passo.

Empírico: o tempo cresce moderadamente ($0,023\text{s} \rightarrow 0,037\text{s}$ no vetor, $+61\%$), menos que no experimento booleano, pois `faixa_rec` visita apenas nós compatíveis com o intervalo, sem materializar o universo inteiro. Vetor e hash permanecem próximos, confirmando que o acréscimo vem do índice AVL comum, não da tabela de buckets.

5.2.8 Histórico agregado: `HistoricoLista` vs `HistoricoMatriz`

Experimento **H01**: $U = P = 1000$, $R = 200$, 100 consultas LU e 100 LP, 2 filtros, 3 itens por compra; variando $C \in \{50, 100, 200, 400, 800\}$. Índices fixos em `MapaVetor`; compara-se `tp3.out` (lista) com `tp3.hist_matriz.out` (matriz).

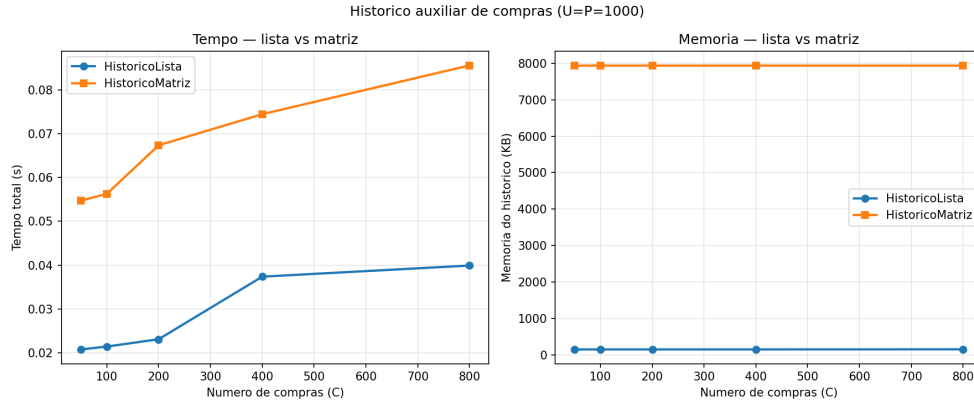


Figura 8: Tempo total e memória do histórico, `HistoricoLista` vs `HistoricoMatriz`.

Teoria: `HistoricoLista` atualiza em $O(d)$ e imprime LU/LP em $O(d)$; `HistoricoMatriz` atualiza em $O(1)$ por célula, mas imprime em $O(P)$ ou $O(U)$ (Tabela 2). Memória: $O(\sum d_u + \sum d_p)$ vs. $\Theta(U \cdot P)$.

Empírico: a memória da matriz permanece em $\approx 7,8\text{ MiB}$ ($\Theta(U \cdot P)$, independente de C); a lista varia de $\approx 157\text{ KiB}$ a $\approx 161\text{ KiB}$ — esparsa, ordens de magnitude abaixo da matriz. O tempo da lista ($0,021\text{--}0,040\text{s}$) escala sublinearmente com C ($O(q \cdot d)$ por compra com d pequeno); o da matriz ($0,055\text{--}0,086\text{s}$) é quase constante na atualização $O(q)$, mas LP penaliza a impressão: $\approx 52\mu\text{s}$ por consulta na lista vs. $\approx 146\mu\text{s}$ na matriz com $C = 800$, ratio $\approx 2,8\times$ coerente com $O(d)$ vs. $O(U)$ na segunda linha de saída.

Localidade vs custo: a matriz concentra quantidades em arrays contíguos, mas penaliza cargas esparsas.

5.2.9 Localidade de referência: medição experimental

Conforme a seção 2.1 do enunciado, complementamos os experimentos de tempo e memória com duas ferramentas do Valgrind: **Cachegrind** (taxas de miss nas caches simuladas, proxy quantitativo de localidade) e **Lackey** (mapa de acesso endereço $\times$ ciclo, ilustrando acessos sequenciais ou reutilização de endereços). Foram considerados três perfis de carga (*cadastros*, *transações* e *consultas*) e, em cada um, as variantes **MapaVetor** e **MapaHash** nos índices invertidos (*seed=42*).

Taxas de miss de cache (Cachegrind). O Cachegrind simula caches L1 de dados (D1) e último nível (LLd). Quanto *menor* a taxa de miss, melhor a localidade de referência naquele perfil. A Tabela 3 resume os resultados; a Figura 9 exibe as barras por perfil.

Perfil	MapaVetor D1	Hash D1	Observação
Cadastros ($U = P = 1000$)	0,20%	0,10%	Hash: menos deslocamentos no vetor ordenado
Transações ($C = R = 400$)	0,10%	0,10%	Empate; domina <code>IndiceOrdenado</code>
Consultas (400 de cada tipo)	0,10%	0,10%	Vetor: menos misses absolutos

Tabela 3: Taxa de miss na cache L1 de dados (D1) — `MapaVetor` vs Hash.

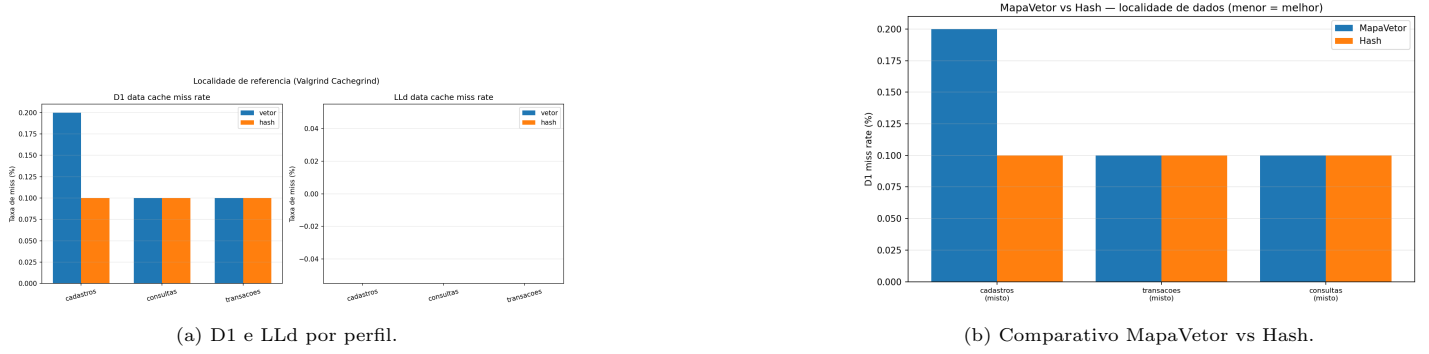


Figura 9: Localidade quantificada via Valgrind Cachegrind.

As taxas absolutas são baixas ($< 0,2\%$ em D1), pois o Valgrind simula caches generosas e a carga mista acessa repetidamente as mesmas regiões. Ainda assim, o `cg_annotate` revela diferenças *por função*: em cadastros, `ConjuntoIds::inserir` e `MapaVetorInt::buscar_pos` concentram leituras sequenciais no vetor de entradas; na hash, `MapaHashInt::redimensionar` e a travessia de colisões introduzem saltos entre nós alocados separadamente. Em consultas, `IndiceOrdenado::faixa_rec` aparece entre as funções com maior taxa de D1 miss relativa em *ambas* as variantes — reflexo da árvore balanceada com nós dispersos no *heap*, independente do backend do mapa invertido.

Mapas de acesso à memória (Lackey). Cada ponto (x, y) do mapa representa um *load* de dados: x é o índice sequencial do acesso (ciclo) e y é o endereço normalizado. Com isso:

- faixas **diagonais** $\Rightarrow$ **localidade espacial** endereço cresce com o ciclo, ou seja, varredura sequencial em array contíguo;
- faixas **horizontais** $\Rightarrow$ **localidade temporal** o mesmo endereço, ou faixa estreita, é reutilizado em ciclos consecutivos.

Para isolar os padrões sem ruído de E/S, executamos cargas *demo* que exercitam cada TAD (`bench/localidade_demo.cpp`): interseção repetida em `ConjuntoIds`, varredura linear em `Vetor<int>` e buscas em `MapaVetorInt`. A Figura 10 destaca os dois tipos de localidade.

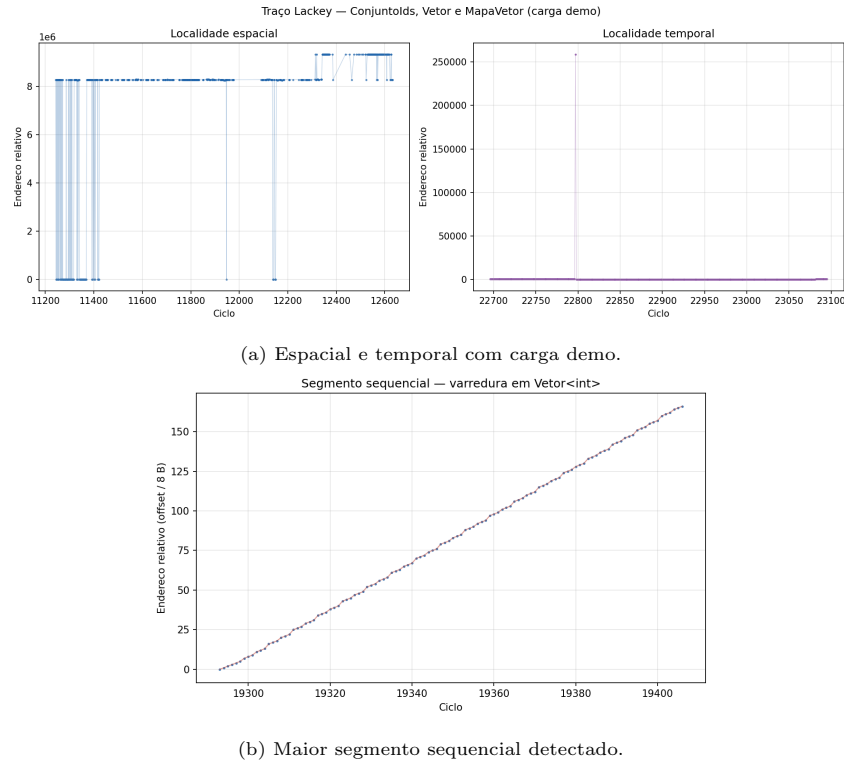
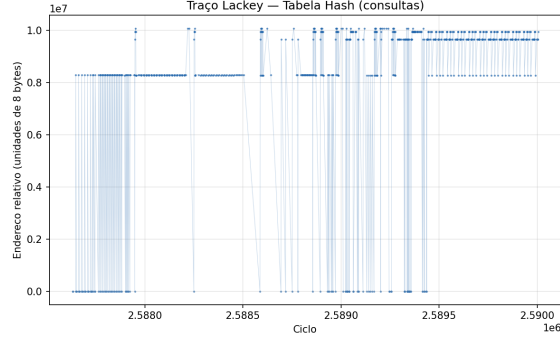


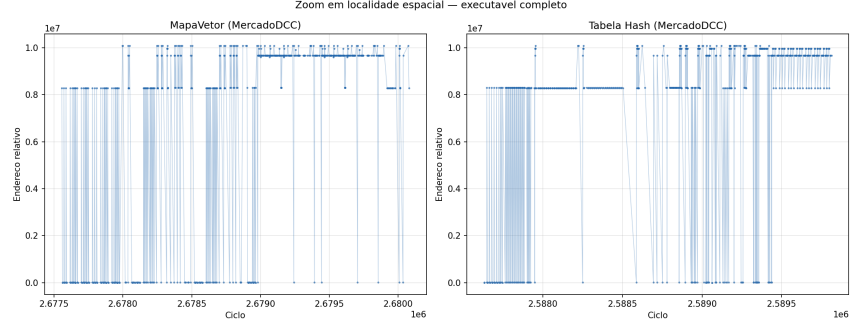
Figura 10: Mapa endereço $\times$ ciclo, padrões de localidade espacial e temporal.

No painel esquerdo, a faixa diagonal corresponde à varredura de `Vetor<int>` e à interseção two-pointer em `ConjuntoIds`; a faixa horizontal evidencia variáveis de laço e conjuntos reutilizados entre iterações. O painel direito amplia o maior *run* sequencial detectado automaticamente, endereços crescentes em passos de até 64 bytes, típico da percorrer `_ids[i]`, `_ids[i+1]`, ... durante interseção ou impressão ordenada.

Índices invertidos no executável completo. Repetiu-se o traço Lackey no binário completo, descartando os primeiros $\approx 42\%$ dos loads, fase de cadastro, e amostrando 35 000 loads da fase de consultas com $U = P = 60$, 40 consultas de cada tipo. A Figura 11 compara MapaVetor e Hash, com zoom espacial no painel direito.



(a) Tabela Hash (consultas).

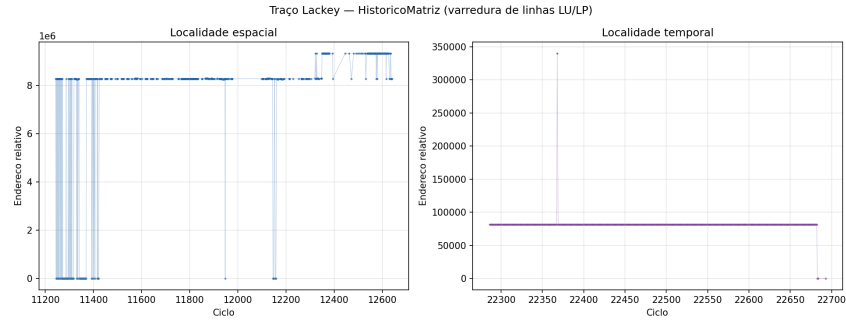


(b) Zoom espacial — vetor vs hash.

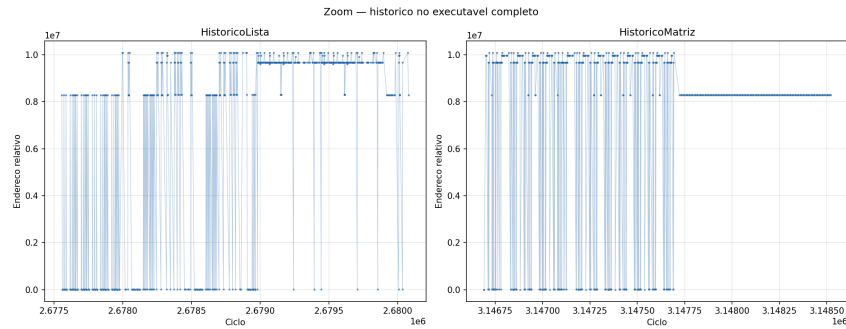
Figura 11: Traço Lackey no *MercadoDCC* completo fase de consultas.

No **MapaVetor**, buscas binárias e interseções produzem diagonais mais contíguas: cada filtro percorre entradas adjacentes no vetor ordenado antes de combinar os **ConjuntoIds**. Na **hash**, o array de buckets é contíguo, faixas horizontais curtas, mas cada colisão salta para nó encadeado em endereço distinto: a nuvem de pontos torna-se mais espalhada verticalmente, coerente com a menor localidade espacial descrita na Seção 4.

Histórico agregado: lista vs matriz. A Figura 12 contrasta *HistoricoLista* e *HistoricoMatriz* na mesma carga de consultas LU/LP demo à esquerda; executável completo à direita.



(a) Carga demo (matriz).



(b) Lista vs matriz (completo).

Figura 12: Mapas de acesso — histórico de compras.

A **matriz** gera diagonais longas ao varrer a linha inteira `_usuario_produto[u][0..P-1]` na segunda linha de LU/LP, forte localidade espacial, porém sobre células muitas vezes zeradas. A **lista** toca menos endereços distintos, ou seja, faixas mais estreitas, mas alterna entre listas alocadas separadamente quando o fluxo muda de usuário para produto, produzindo saltos irregulares. Isso explica o paradoxo observado no experimento HistóricoLista vs HistoricoMatriz: melhor localidade espacial na matriz, porém pior tempo quando o histórico é esparsos.

Estrutura	Padrão de acesso	Evidência experimental
ConjuntoIds	Sequencial em <code>_ids</code> (interseção two-pointer)	Diagonais na demo; baixa D1 miss em <code>operator[]</code> e <code>intersect</code>
MapaVetor	Bloco contíguo de entradas; busca binária em região compacta	Diagonais contíguas em consultas; D1 ligeiramente maior em cadastros (deslocamento na inserção)
MapaHash	Buckets contíguos + nós encadeados dispersos	Horizontais nos buckets; dispersão vertical nas colisões; D1 menor em cadastros
IndiceOrdenado	Saltos por ponteiro na AVL	Alta D1 miss relativa em <code>faixa_rec</code> (consultas com faixa)
Vetor<Entidade>	Acesso direto por id	Acessos adjacentes quando ids consecutivos no resultado da consulta
HistoricoMatriz	Varredura contígua de linha	Diagonais longas; mais dados lidos
HistoricoLista	Listas contíguas por usuário/produto, dispersas entre si	Menos endereços; saltos entre listas quando alterna entidades

Tabela 4: Correlação estrutura $\times$ padrão de acesso à memória.

Síntese por estrutura de dados. Em conjunto, as medições confirmam a análise qualitativa da Seção 4: estruturas com arrays contíguos (`ConjuntoIds`, `MapaVetor`, `HistoricoMatriz`) favorecem acessos sequenciais visíveis como diagonais nos mapas Lackey; estruturas com indireção por ponteiro (`MapaHash`, `IndiceOrdenado`, nós de lista no histórico) aumentam a dispersão e, em cadastros intensos, podem elevar misses mesmo com complexidade assintótica favorável.

5.3 Síntese experimental: teoria vs. medição

A Tabela 5 cruza previsão assintótica e evidência observada; a Tabela 6 resume a variante favorecida por cenário.

Experimento	Previsão (Seção 3)	Observado
E01/E02 cadastros	Tempo $O(U \cdot k)$ vetor; $O(U)$ hash; memória linear	Curvas $\nearrow$ lineares; hash $\approx 3\times$ em $U, P = 4000$
E03 transações	$O(C)$ marginal, diluído por carga fixa	Tempo +100% de 50 a 800 transações
E04 filtros	$O(f \cdot m)$; m decrescente	Tempo quase plano ($f = 1..5$)
E05 consultas	Comparações $\propto Q$; lookup secundário	Cmp. $\times 5,2$; tempo $\times 1,6$ (100 $\rightarrow$ 500)
E06 booleanos	Complemento $O(n)$; união $O(m)$	Tempo $\times 3,6$; cmp. $\times 2,2$
E07 faixas	<code>faixa_rec</code> $O(\log k + t)$	Tempo +61%; empate vetor/hash
H01 histórico	Lista $O(d)$ impressão; matriz $\Theta(UP)$ mem.	Lista $\ll$ memória; LP $2,8\times$ mais rápida

Tabela 5: Cruzamento entre complexidade teórica e resultados empíricos.

Cenário dominante	Tendência	Variante favorecida
Cadastros massivos (U ou P altos)	Tempo $\uparrow$ linear; memória $\uparrow$ linear	Hash (tempo)
<code>_usuarios</code>	Tempo $\uparrow$ linear; memória $\uparrow$ linear	Neutro (índices dominam)
Muitas transações (C, R)	Tempo $\uparrow$; memória histórico $\uparrow$	Empate
Muitos filtros / consultas	Comparações em conjuntos $\uparrow$	Empate
Consultas booleanas	Tempo $\uparrow$ até $\sim 3,6\times$ (complemento $O(n)$)	Hash (leve)
Faixas preço/qtd	Tempo $\uparrow$ moderado	Empate
Memória total (índices)	Hash $\approx 17\%$ maior na baseline	MapaVetor
Histórico esparsa ($U = P = 1000$)	Memória lista $\ll$ matriz; tempo lista $<$ matriz	HistoricoLista
Cache D1 (cadastros)	Vetor 0,20% vs Hash 0,10%	Hash
Traço Lackey (consultas)	Matriz: diagonais longas; hash: mais dispersão	MapaVetor / Lista

Tabela 6: Resumo qualitativo dos experimentos isolados.

Nenhuma estrutura vence em todas as dimensões. Em conjunto, os experimentos validam a Seção 3: diferenças vetor/hash aparecem onde a teoria prevê ($O(k)$ vs. $O(1)$ em cadastros); empates aparecem onde ambos compartilham `ConjuntoIds` e `IndiceOrdenado` (consultas puras, faixas); o histórico esparsa confirma o trade-off $O(d)$ vs. $\Theta(U \cdot P)$ da Tabela 2. O **MapaVetor** consome menos memória; a **hash** reduz tempo em cadastros intensos; a **lista** domina no histórico esparsa.

6 Conclusão

A principal lição deste trabalho é que a escolha de estruturas de dados depende do *perfil de carga*, e não de uma hierarquia universal de “melhor” implementação. Índices invertidos sobre `ConjuntoIds` ordenados mostraram-se um padrão eficaz para consultas multiatributo: cada filtro reduz o universo a um conjunto de identificadores e a interseção linear reutiliza a ordenação como invariante. Manter backends intercambiáveis (`MapaVetor/MapaHash`, `HistoricoLista/HistoricoMatriz`) permitiu isolar o efeito de cada decisão estrutural sem alterar a lógica de negócio.

Por fim, aprendeu-se a implementar uma tabela hash e que invariantes explícitos: como estoque validado antes de alterar estado, reindexação de quantidade, conjuntos sem duplicatas, complemento booleano relativo ao universo são tão importantes quanto a eficiência assintótica para manter o sistema correto sob atualizações parciais. Nenhuma variante vence em tempo, memória e localidade simultaneamente; a decisão prática deve combinar previsão analítica, medição empírica e conhecimento do cenário de uso como cadastros intensos, consultas com muitos filtros, histórico esparsa ou denso. Para o perfil do *MercadoDCC*, **MapaVetor** com **HistoricoLista** equilibra memória e desempenho; a hash e a matriz permanecem alternativas justificáveis quando cadastros ou densidade do histórico dominam o custo.

7 Bibliografia

Referências

- [1] Anísio Lacerda, Gisele Lobo, Wagner Meira Jr., Washington Cunha. *Estruturas de Dados – Trabalho Prático 3*, DCC221, ICEx/UFMG, 2026.
- [2] Anísio Lacerda, Gisele Lobo, Wagner Meira Jr., Washington Cunha. *Estruturas de Dados*, slides da disciplina DCC221, Departamento de Ciência da Computação, ICEx/UFMG.
- [3] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.